Pipeline Recovery Internals

The short Pipeline Recovery guide ran a plan, caught a draft that drifted, and retried from the last good boundary. This notebook rebuilds that run and opens it up: a plan held as a logical boundary, a failed draft, a reviewer and an inspector, durable artifact refs, and a retry that CITES the retained plan instead of writing into it. Along the way it reveals the retained run record, the durable argument refs, and a regenerated trace projection.

Setup

Load the launch helpers.

In [1]:
```
import pathlib
import sys


def _find_visual_artifact_example_root():
    cwd = pathlib.Path.cwd().resolve()
    candidates = []
    for base in (cwd, *cwd.parents):
        candidates.append(base)
        candidates.append(base / "examples" / "notebooks" / "visual_artifact
    for candidate in candidates:
        if (candidate / "shepherd_usecases" / "visual_artifact" / "launch.py
            return candidate
    raise RuntimeError(
        "Cannot find examples/notebooks/visual_artifact. "
        "Launch JupyterLab from the repository root with `make notebooks`."
    )


example_root = _find_visual_artifact_example_root()
if str(example_root) not in sys.path:
    sys.path.insert(0, str(example_root))
try:
    from shepherd_usecases.visual_artifact import launch
    from shepherd_usecases.visual_artifact import viz
except Exception as exc:
    raise RuntimeError(
        "Could not import the visual-artifact notebook helpers. "
        "Launch JupyterLab from the repository root with `make notebooks`."
    ) from exc

launch.bootstrap(example_root=example_root)
```

Build a run to inspect

Start by building the run this notebook takes apart. `launch.plan_for` turns the prompt into a brief and a plan; `launch.open_workspace` opens one flow to hold every run. The cells below run each step by name and read them back to draw the trace and to drive the retry.

In [2]:
```
prompt = launch.default_prompt()
brief, plan = launch.plan_for(prompt)
plan
```

Out[2]:
```
{'artifact': 'gradient_descent_tile',
 'framing': 'contour-map',
 'request': 'Please output an infographic tile explaining gradient descen
t.',
 'must_include': ['Gradient Descent',
  'Start',
  'Step',
  'Minimum',
  'Small downhill steps reduce error.'],
 'decision_critical': 'update path must descend toward the minimum'}
```

In [3]:
```
workspace = launch.open_workspace("visual-pipeline-recovery-internals", prom
```

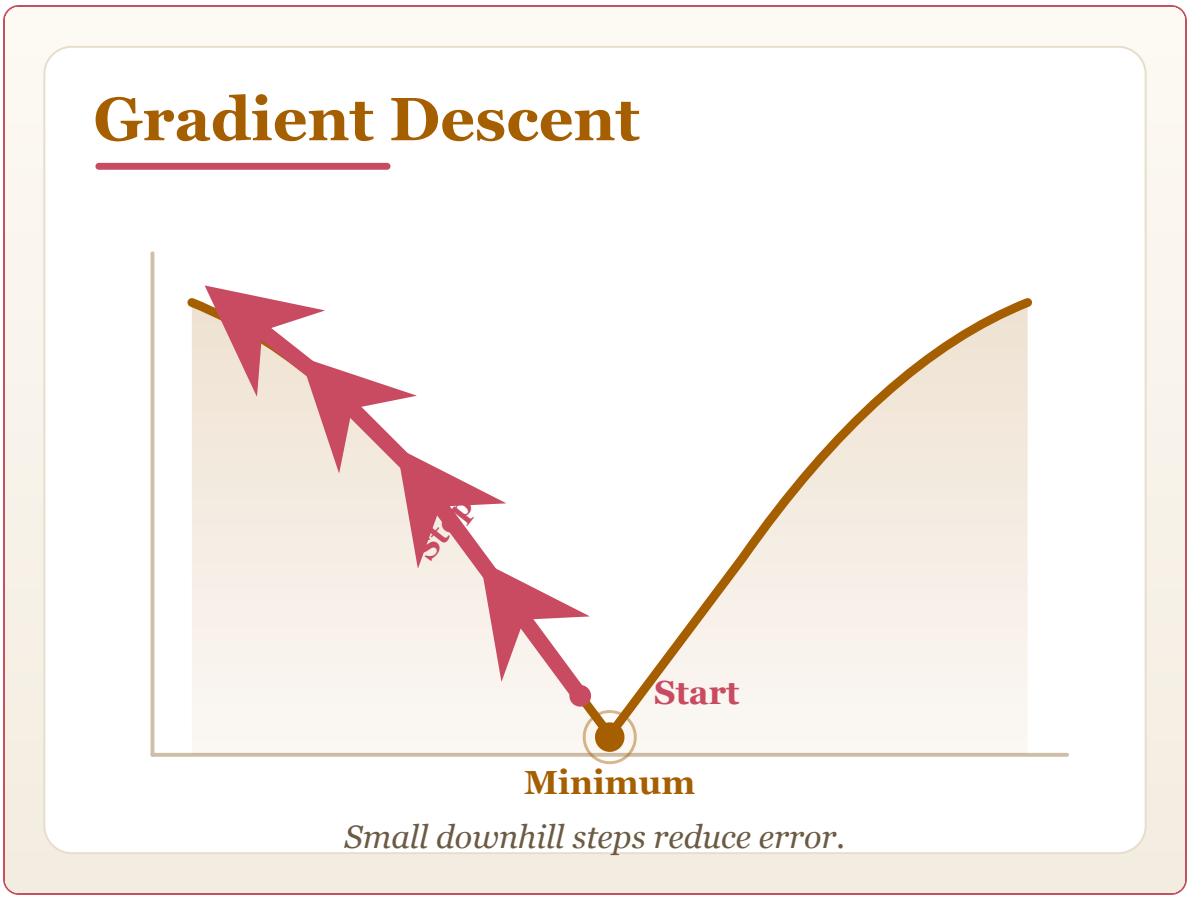
The plan boundary and the draft that drifted

The pipeline is two runs made in order. `plan_run` writes the plan and stands as the logical boundary; `launch.artifact_ref` freezes a durable citation to it. The draft run follows `after=[plan_run]` and cites that plan ref, so it builds on the plan instead of starting from scratch. `corrupt=True` makes this draft drift uphill, away from the minimum. The failed artifact renders in red.

In [4]:
```
plan_run = launch.run_static(
    workspace,
    name="plan",
    output_path=launch.PLAN_PATH,
    output_content=plan,
    metadata={"logical_boundary": "plan"},
)
plan_ref = launch.artifact_ref(plan_run, launch.PLAN_PATH, label="retry-plar

draft_v1 = launch.run_with_artifact_ref(
    workspace,
    name="draft-v1",
    ref_name="plan",
    artifact_ref=plan_ref,
    output_path=launch.ARTIFACT_PATH,
    output_text=launch.draft_html(brief, corrupt=True),
    after=[plan_run],
    metadata={"failed_run": "draft-v1"},
)
viz.show(viz.run_artifact(draft_v1, label="draft v1: drifts uphill", accent=
```

draft v1: drifts uphill



The reviewer and inspector read what the draft recorded

Neither run re-executes the draft; each cites a retained artifact and reads it. The reviewer cites the failed draft and writes a verdict; `launch.selection_from_review` reads that verdict back and the draft's issues fall out of it. The inspector then cites the review, classifies the drift with `launch.diagnosis_content`, and writes a diagnosis. Citing the runs as durable refs is what lets each read the other's output without running anything again.

In [5]:
```
draft_ref = launch.artifact_ref(draft_v1, label="failed-draft")
reviewer = launch.run_with_artifact_ref(
    workspace,
    name="review",
    ref_name="candidate",
    artifact_ref=draft_ref,
    output_path=launch.VERDICT_PATH,
    output_content=launch.review_content(prompt, {"draft_v1": draft_v1}),
    after=[draft_v1],
)
selection = launch.selection_from_review(reviewer)
issues = list(selection.candidates[0].get("issues", []))
review_ref = launch.artifact_ref(reviewer, launch.VERDICT_PATH, label="draft
```

```
inspector = launch.run_with_artifact_ref(
    workspace,
    name="inspector",
    ref_name="review",
    artifact_ref=review_ref,
    output_path=launch.DIAGNOSIS_PATH,
    output_content=launch.diagnosis_content(issues),
    after=[reviewer],
)
launch.read_json(inspector, launch.DIAGNOSIS_PATH)
```

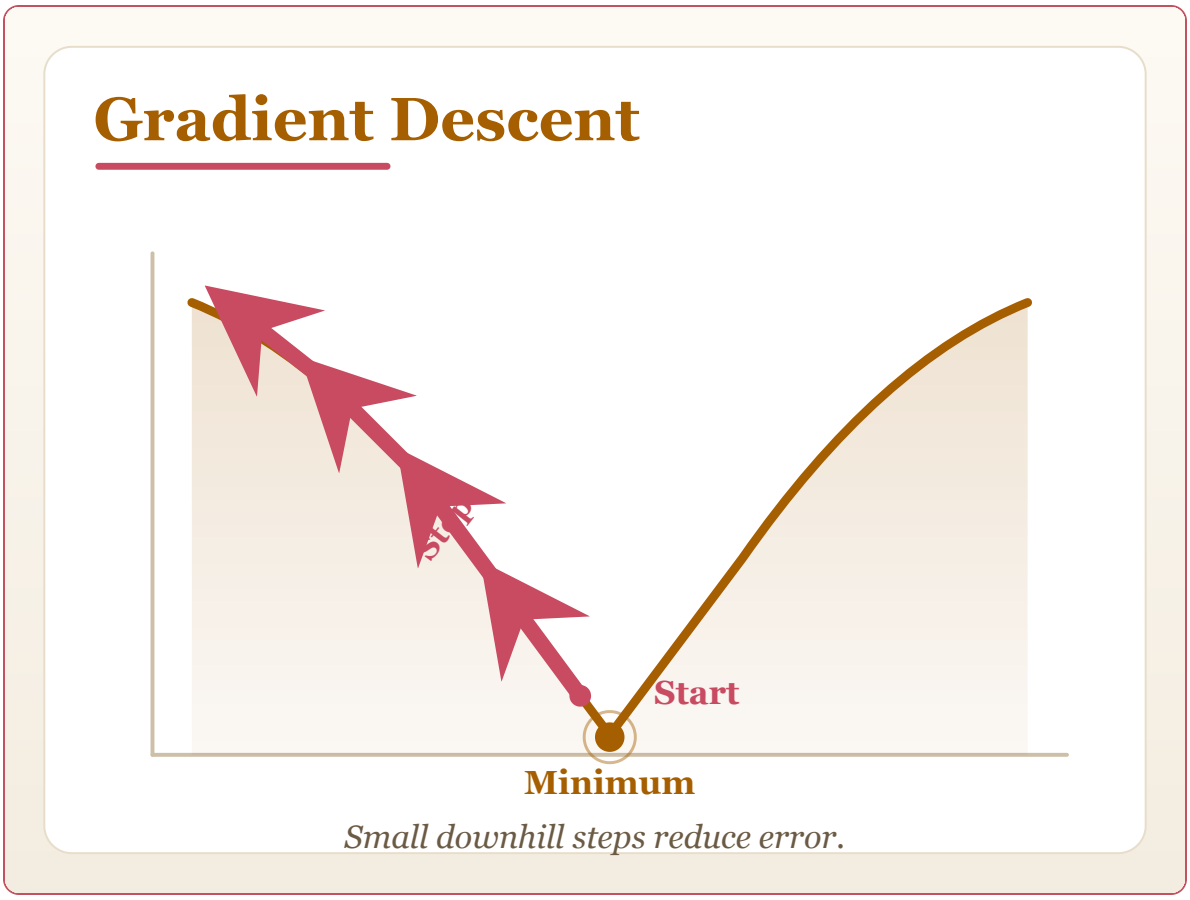
Out[5]:
```
{'bad_step': 'draft',
 'evidence': 'index.html -> descent path moves uphill away from the minimu
m',
 'failure_type': 'wrong_direction',
 'recommended_change': 'Reverse the update path so each step descends towar
d the minimum.',
 'retry_boundary': 'after plan, before draft'}
```

The retry cites the retained plan

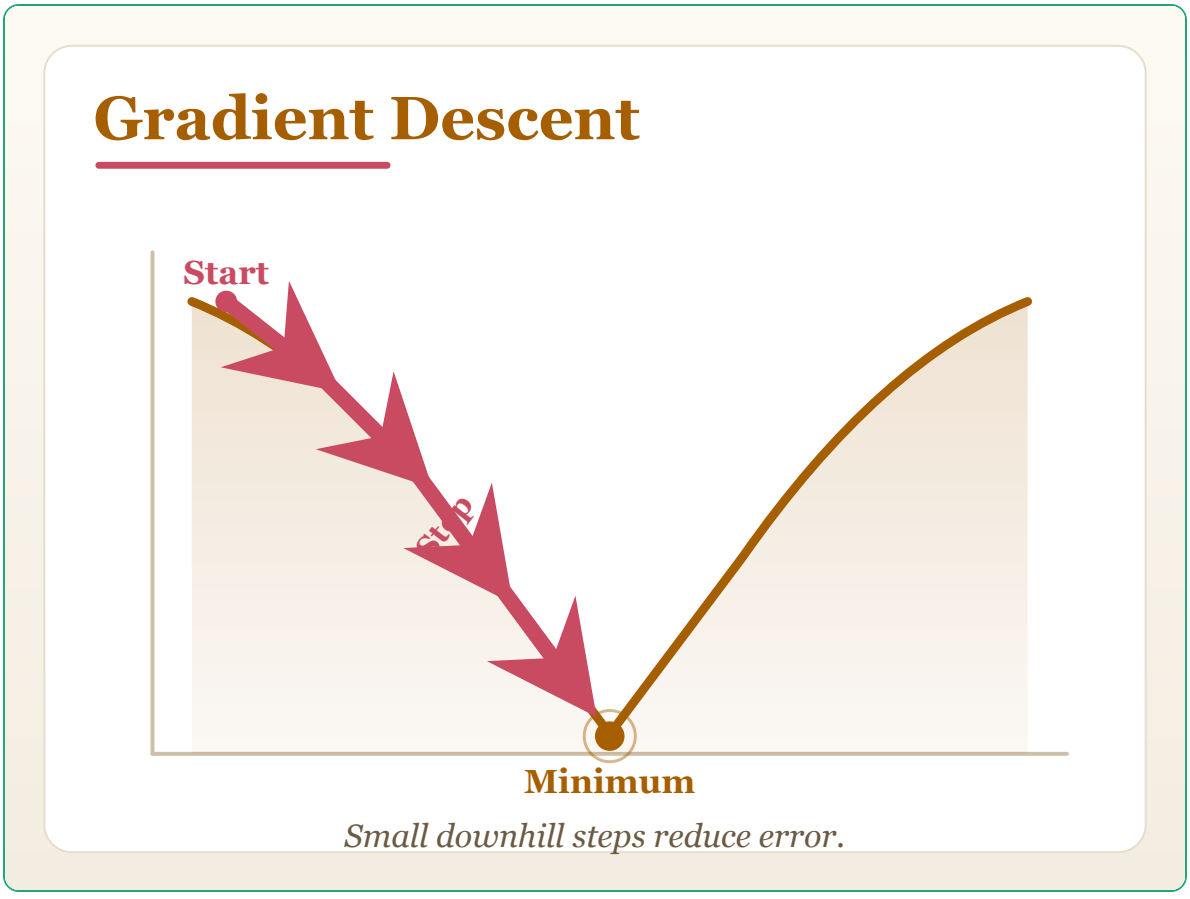
The retry is a NEW run. It cites the retained plan ref and the diagnosis ref, follows `after=[plan_run, inspector]`, and renders `corrupt=False` so it lands back on track. The plan is never reopened as a writable branch: the retry follows it by citation. The failed draft and the retry render side by side, red and green.

In [6]:
```
diagnosis_ref = launch.artifact_ref(inspector, launch.DIAGNOSIS_PATH, label=
retry = launch.run_with_artifact_refs(
    workspace,
    name="retry",
    refs={"plan": plan_ref, "diagnosis": diagnosis_ref},
    output_path=launch.ARTIFACT_PATH,
    output_text=launch.draft_html(brief, corrupt=False),
    after=[plan_run, inspector],
    metadata={"retry_run": "retry-from-plan"},
)
viz.show(viz.side_by_side([
    viz.run_artifact(draft_v1, label="failed draft", accent="red"),
    viz.run_artifact(retry, label="retry from cited plan", accent="green"),
]))
```

failed draft



retry from cited plan



What the retry actually is

The retry's structure lives in its retained run record and durable args, not in any mutated output. `launch.run_record` returns the record with its `args_ref` and `args_digest`; `launch.run_args` resolves the durable arg payload so its input refs can be counted, one per cited artifact. `launch.changed_paths` shows exactly what the retry wrote, which is a fresh artifact, not an edit of the plan.

In [7]:
```
record = launch.run_record(workspace, retry)
args = launch.run_args(workspace, retry)
{
    "run_ref": retry.run_ref,
    "args_ref": record.args_ref,
    "args_digest": record.args_digest,
    "input_ref_count": len(args["input_refs"]),
    "changed_paths": launch.changed_paths(retry),
}
```

Out[7]:
```
{'run_ref': 'run-a4549d5645f2',
 'args_ref': 'run-a4549d5645f2:args:1f10ea56d895ca3e',
 'args_digest': 'sha256:1f10ea56d895ca3eebb449dc038f4cf68f2cb429a167e715b27
2917a0e7a6ad3',
 'input_ref_count': 2,
 'changed_paths': ('index.html',)}
```

Flow trace

`workspace.flow.trace()` returns the flow's projection, plan to retry. `viz.flow_trace` renders it as a self-contained table of events and edges. Follow the edges: the retry links back to the plan and the inspector, and the failed draft stays recorded as evidence.

In [8]:
```
viz.show(viz.flow_trace(workspace.flow.trace(), height="680px"))
```

Events

kind	run/flow	label	payload
flow.opened	flow-cdd8c7ea3b15		name='visual-pipeline-recovery-int
flow.fork.requested	run-5d5702d8c45b		name='plan', after=[]
flow.logical_boundary	run-5d5702d8c45b		label='plan'
run.lifecycle	run-5d5702d8c45b		task_id='shepherd_usecases.visual_ task_version='v1', status='retaine
provider.invocation	run-5d5702d8c45b		execution_id='task-execution-19143 invocation_id='static:b14eb555a2a3 status='started', source='task_exe evidence_role='provider_provenance payload={'args_digest': 'sha256:b5953c66b8b084e3ea13273525 'artifact_path': 'plan.json', 'tas 'shepherd_usecases.visual_artifact
provider.invocation	run-5d5702d8c45b		execution_id='task-execution-19143 invocation_id='static:b14eb555a2a3 provider_event_kind='provider.invo source='task_execution.metadata.pr event_id='static:b14eb555a2a3:comp 'plan.json', 'content_digest': 'sha256:51b24e3c80b7c0ded3f33a43c7 'launched_confined': False}
run.output.published	run-5d5702d8c45b		output_name='workspace', output_id output:2971e398630ac0fbca6464c97be binding='workspace', output_world_
flow.fork.requested	run-dbfc66e6c988		name='draft-v1', after=['run-5d570
flow.failed_run	run-dbfc66e6c988		label='draft-v1'
run.lifecycle	run-dbfc66e6c988		task_id='shepherd_usecases.visual_ task_version='v1', status='retaine
provider.invocation	run-dbfc66e6c988		execution_id='task-execution-0691a invocation_id='static:3c5adf21de8

Task contract

Every run above went through one provider-owned task. `viz.task_contract` shows that task's public contract, its inputs and docstring, without the provider fixture body.

In [9]:
```
from shepherd_usecases.visual_artifact import tasks
viz.task_contract(tasks.static_artifact_task)
```

Out[9]:
```
def static_artifact_task(
    repo: object,
    *,
    output_path: str,
    output_text: str | None = None,
    output_content: object | None = None,
    **artifact_refs: object,
) -> object:
    """Declare the static visual-artifact task executed by the provider runtime."""
    ...
```

Cleanup

Retention is expressed with the output verbs, not by overwriting anything. The failed draft is discarded, the plan, review, and diagnosis are released, and the retry is selected as the kept result. `workspace.close()` releases the flow. Nothing above was overwritten: the failed run, its trace, and the diagnosis all remain, which is what let the inspector work from recorded output in the first place.

In [10]:
```
draft_v1.output().discard()
plan_run.output().release()
reviewer.output().release()
inspector.output().release()
retry.output().select()
workspace.close()
```